Matrices
Strassen's algorithm

George Mertzios
george.mertzios@durham.ac.uk

Algorithms and Complexity

Lecture 4

Outline

Master Method

Divide-and-conquer for matrix multiplication

Strassen's algorithm

Learning outcomes:

1. to be able to use the master method to solve recurrence relations
2. to be able to explain Strassen's algorithm
3. to be able to explain divide and conquer

Divide-and-conquer algorithms

Recall that many useful algorithms are recursive, i.e. call themselves recursively to deal with subproblems.

At *each* level of recursion:

- Divide the problem into a number of subproblems.
- Conquer the subproblems by solving them: recursively, or straightforward if the subproblem sizes are small enough.
- Combine the solutions for the subproblems into a solution for the original problem.

Examples: MERGE-SORT, QUICKSORT and from Lecture 3: Graph Coloring for Cographs, Maximum Subarray, etc.

Master method

The master method depends on the master theorem.

Let $T(n)$ be a function on the natural numbers defined by

$$T(n) = aT(n/b) + f(n).$$

(Interpret n/b as either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$.)

We will compare $f(n)$ with the function $n^{log_b a}$.

The master theorem

Let $f(n)$ be a function on the natural numbers, and let $T(n)$ be defined by

$$T(n) = aT(n/b) + f(n),$$

for some constants $a \geq 1$ and $b > 1$. Then:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.
3. If $f(n) = \Omega(n^{log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $a \cdot f(n/b) \leq c\dot{f}(n)$ for some constant $c < 1$ and for sufficiently large n, then $T(n) = \Theta(f(n))$.

Applying the master theorem

Example: MERGE-SORT satisfies

$$T(n) = 2T(n/2) + \Theta(n)$$

(since we sort both parts separately in $2T(n/2)$ and we merge them in $\Theta(n)$).

We are in Case 2, hence we obtain $T(n) = \Theta(n \log n)$ for MERGE-SORT.

- The same for MAXIMUM SUBARRAY.

Multiplying two matrices

We are interested in the following problem:

- input: two square matrices $A, B \in \mathbb{R}^{n \times n}$
- output: their product $C \in \mathbb{R}^{n \times n}$

We can do it in time $\Theta(n^3)$ by using the definition: (Why?)

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}.$$

Simple Divide-and-conquer approach

(For simplicity, we assume n is a power of two.)

Partition A, B, and C into four parts (called block matrices):

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Then

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22}. \end{aligned}$$

Simple Divide-and-conquer approach: Pseudo-code

Algorithm 1 MULT(A, B)

```
let C be a new n × n matrix
if n = 1 then
    c_11 ← a_11 b_11
else
    partition A, B, and C into the block matrices
    C_11 ← MULT(A_11, B_11) + MULT(A_12, B_21)
    C_12 ← MULT(A_11, B_12) + MULT(A_12, B_22)
    C_21 ← MULT(A_21, B_11) + MULT(A_22, B_21)
    C_22 ← MULT(A_21, B_12) + MULT(A_22, B_22)
return C
```

Running time of simple approach

The running time of MULT(A, B) satisfies

$$T(n) = 8T(n/2) + \Theta(n^2),$$

since we have:

- 8 multiplications of $\frac{n}{2} \times \frac{n}{2}$ matrices, and
- 4 matrix additions (each taking time $\Theta(\frac{n}{2} \cdot \frac{n}{2}) = \Theta(n^2)$).

The Master theorem implies: $T(n) = \Theta(n^3)$ (check this!)

Strassen's algorithm

Strassen (1969) introduced a sophisticated divide-and-conquer approach. It has four steps:

1. Divide A, B, and C into four parts as before.
   Running time: $\Theta(1)$ (using appropriate index calculations).
2. Create 10 auxiliary matrices $S_1, \ldots, S_{10} \in \mathbb{R}^{n/2 \times n/2}$, obtained by sums or differences of $A_{11}, \ldots, B_{22}$.
   Running time: $\Theta(n^2)$.
3. Using all the matrices available, recursively compute **seven** product matrices $P_1, \ldots, P_7 \in \mathbb{R}^{n/2 \times n/2}$.
   Running time: $7T(n/2)$.
4. Compute $C_{11}, \ldots, C_{22}$ by adding / subtracting combinations of $P_1, \ldots, P_7$.
   Running time: $\Theta(n^2)$.

Running time of Strassen's

Thus, the worst-case running time of Strassen's algorithm satisfies

$$T(n) \leq 7T(n/2) + \Theta(n^2).$$

By the master method we obtain

$$T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.8074}).$$

Description of $S_1, \ldots, S_{10}$

$$
\begin{aligned}
S_1 &= B_{12} - B_{22} \\
S_2 &= A_{11} + A_{12} \\
S_3 &= A_{21} + A_{22} \\
S_4 &= B_{21} - B_{11} \\
S_5 &= A_{11} + A_{22} \\
S_6 &= B_{11} + B_{22} \\
S_7 &= A_{12} - A_{22} \\
S_8 &= B_{21} + B_{22} \\
S_9 &= A_{11} - A_{21} \\
S_{10} &= B_{11} + B_{12}
\end{aligned}
$$

Description of $P_1, \ldots, P_7$

$$
\begin{aligned}
P_1 &= A_{11}S_1 = A_{11}B_{12} - A_{11}B_{22} \\
P_2 &= S_2B_{22} = A_{11}B_{22} + A_{12}B_{22} \\
P_3 &= S_3B_{11} = A_{21}B_{11} + A_{22}B_{11} \\
P_4 &= A_{22}S_4 = A_{22}B_{21} - A_{22}B_{11} \\
P_5 &= S_5S_6 = A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} \\
P_6 &= S_7S_8 = A_{12}B_{21} + A_{12}B_{22} - A_{22}B_{21} - A_{22}B_{22} \\
P_7 &= S_9S_{10} = A_{11}B_{11} + A_{11}B_{12} - A_{21}B_{11} - A_{21}B_{12}
\end{aligned}
$$

Expressing C

$$
\begin{aligned}
S_1 &= B_{12} - \boxed{B_{22}} \\
S_2 &= \boxed{A_{11}} + \boxed{A_{12}} \\
S_3 &= A_{21} + \boxed{A_{22}} \\
S_4 &= B_{21} - B_{11} \\
S_5 &= A_{11} + A_{22} \\
S_6 &= \boxed{B_{11}} + B_{22} \\
S_7 &= A_{12} - A_{22} \\
S_8 &= \boxed{B_{21}} + B_{22} \\
S_9 &= A_{11} - A_{21} \\
S_{10} &= B_{11} + B_{12}
\end{aligned}
$$

$$
\begin{aligned}
P_1 &= A_{11}S_1 = A_{11}B_{12} - A_{11}B_{22} \\
&\boxed{P_2 = S_2B_{22} = A_{11}B_{22} + A_{12}B_{22}} \\
P_3 &= S_3B_{11} = A_{21}B_{11} + A_{22}B_{11} \\
&\boxed{P_4 = A_{22}S_4 = A_{22}B_{21} - A_{22}B_{11}} \\
&\boxed{P_5 = S_5S_6 = A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22}} \\
&\boxed{P_6 = S_7S_8 = A_{12}B_{21} + A_{12}B_{22} - A_{22}B_{21} - A_{22}B_{22}} \\
P_7 &= S_9S_{10} = A_{11}B_{11} + A_{11}B_{12} - A_{21}B_{11} - A_{21}B_{12}
\end{aligned}
$$

$$
\begin{aligned}
&\boxed{C_{11} = P_5 + P_4 - P_2 + P_6} \\
C_{12} &= P_1 + P_2 \\
C_{21} &= P_3 + P_4 \\
C_{22} &= P_5 + P_1 - P_3 - P_7.
\end{aligned}
$$

An application of Strassen's idea

Professor Guru wishes to develop a matrix-multiplication algorithm that is asymptotically faster than Strassen's algorithm. His algorithm will use the divide-and-conquer method, dividing each matrix into pieces of size $\frac{n}{4} \times \frac{n}{4}$, and the steps divide and combine will take together $\Theta(n^2)$ time.

He needs to determine how many subproblems his algorithm has to create in order to be asymptotically faster than Strassen's algorithm. Can you help him?